

DiagramTec — Documentación del sector Frontend

Proyecto: DiagramTec (editor de diagramas de flujo + clases educativas)

Repositorio: <https://github.com/MaiteVazquez8/DiagramTec>

Carpeta del sector: DiagramTec/frontend/

Fecha de referencia: Mayo 2026

1. Integrantes del equipo

Integrante	Rol en el proyecto	Participación en Git (commits aprox.)
Santiago Alvarez	Frontend / integración editor y clases	16

Completar nombres y roles según el acta del grupo si difieren.

2. Aspectos técnicos implementados

2.1 Arquitectura general

El frontend es una **SPA (Single Page Application)** que consume dos backends en desarrollo:

Capa	Tecnología	Puerto / ruta
UI	React 18 + Vite 5	5173 (dev)
API principal	Node.js + Express	4002 → proxy /api
Autenticación	PHP (Laragon)	proxy /php-auth
Base de datos	MySQL (tecdiagram)	Laragon

2.2 Tecnologías y herramientas del frontend

Herramienta	Versión	Uso en el proyecto
React	18.3	Componentes, estado local, contexto de sesión
React Router	6.15	Rutas públicas, protegidas y rol superadmin
Vite	5.4	Servidor de desarrollo, build de producción, proxy
Axios	1.7	Peticiones HTTP autenticadas (api.js, authApi.js)
Iconify	6.0	Iconografía alineada al diseño Figma (Icon.jsx)
html2canvas	1.4	Captura del lienzo para miniatura y exportación
jsPDF	4.2	Generación de PDF del diagrama
pnpm	—	Gestor de paquetes del frontend

2.3 Backend y base de datos (integración desde el frontend)

Aunque el sector es frontend, la aplicación se apoya en:

- **Node.js (Express):** diseños, clases, comentarios, panel superadmin, JWT en `/auth/me` .
- **PHP:** login, registro, recuperación de contraseña (`authApi.js` → `/php-auth`).
- **MySQL:** tablas `users` , `classes` , `class_members` , `designs` , `comments` (ver `backend/database/schema.sql`).

El frontend **no accede directamente** a la base de datos; todo pasa por APIs REST con token JWT.

2.4 Librerías y módulos propios

Módulo / carpeta	Responsabilidad
src/design/	Lógica del editor: formas, conexiones, zoom, pan, undo/redo, exportación
src/pages/	Pantallas por ruta (editor, clases, diseños, auth, admin)
src/components/	UI reutilizable (editor, tarjetas Figma, <code>ClassPost</code> , <code>AppToast</code>)
src/AuthContext.jsx	Sesión JWT + sincronización de logout entre pestañas
src/styles.css	Design system (variables Figma, editor, clases, modales)

Documentación técnica adicional: [FRONTEND.md](#) .

2.5 Funcionalidades principales desarrolladas

- **Autenticación:** login, registro, recuperar contraseña, perfil y cierre de sesión multi-pestaña.
- **Editor de diagramas:** paleta, lienzo con zoom/pan, conexiones SVG, historial, guardado en nube, PDF.
- **Mis diseños:** grilla de tarjetas estilo Figma, crear/abrir/eliminar diseños.
- **Clases:** vista profesor (tarjetas compactas, crear clase) vs estudiante (unirse por código).
- **Detalle de clase:** banner, publicaciones, subir diseño, expulsar alumno (profesor), menú contextual.
- **Superadmin:** panel administrativo (ruta protegida por rol).
- **UI Figma:** header con patrón de puntos, notificaciones burdeas, modales unificados.

3. Pruebas y depuración

3.1 Metodología de pruebas

- Pruebas **manuales** en entorno local (Laragon + `pnpm run dev`).
- Verificación de flujos por rol: estudiante, profesor, superadmin.
- Pruebas en **PC y móvil** en la misma red Wi-Fi (`vite --host 0.0.0.0`).
- Build de producción: `pnpm run build` para detectar errores de compilación.

3.2 Errores detectados y soluciones

Error / síntoma	Causa	Solución aplicada
Conexiones no visibles en el lienzo	Estilos SVG ausentes; IDs numéricos vs string en el mapa de formas	Estilos en <code>#connections-layer</code> , <code>normalizeConnection</code> y <code>resolveShapeFromMap</code>
<code>preventDefault</code> en touch (consola)	Listener pasivo en el canvas	Ajuste de eventos táctiles en modo conexión / pan

Profesor no podía publicar en clase	Comparación estricta de <code>ownerId</code> ; dueño sin fila en <code>class_members</code>	<code>Number()</code> en comparaciones; auto-alta del dueño en <code>class_members</code>
Vista de clases igual para todos los roles	CSS y JSX sin bifurcar <code>teacher/student</code>	Clases <code>classes-page--teacher / --student</code> y <code>renderClassCard</code> condicional
Logout solo en una pestaña	<code>localStorage</code> no notifica a la misma pestaña	Evento <code>storage</code> en <code>AuthContext.jsx</code>
Campos de modal con texto en mayúsculas	Estilos globales de <code>.modal-body input</code>	Overrides en <code>.modal--figma-form</code>
Backend no arrancaba / DB	Variables <code>.env</code> o esquema incompleto	Script <code>seed_admin.js</code> , documentación de <code>.env</code> , merge de rama Backend
Acceso móvil a Vite	IP de VirtualBox en lugar de Wi-Fi	<code>host: '0.0.0.0'</code> y uso de IP LAN en <code>vite.config.js</code>

3.3 Compatibilidad probada

Navegadores

Navegador	Versión probada	Resultado
Google Chrome	Reciente	OK (referencia principal)
Microsoft Edge	Reciente	OK (Chromium)
Mozilla Firefox	Reciente	OK en flujos principales
Safari (iOS)	Móvil	OK con pruebas táctiles en editor y clases

Dispositivos

Dispositivo	Uso probado
PC Windows	Desarrollo y pruebas completas
Smartphone (Android / iOS)	Navegación, clases, editor básico (zoom, conexión táctil)
Tablet	Pendiente de documentar por el equipo (<i>opcional</i>)

Resoluciones de pantalla

Resolución / breakpoint	Comportamiento
$\geq 1400\text{px}$	Grilla de clases (3 columnas), editor con sidebar completo
992px – 1399px	Grilla 2–3 columnas, toolbar adaptada
$\leq 768\text{px}$	Menú hamburguesa en header, 1–2 columnas en clases, editor en columna
$\leq 480\text{px}$	Tarjetas a 1 columna, modales al 92% del ancho

Breakpoints definidos en `styles.css` (`@media`).

4. Seguridad y validaciones

4.1 Autenticación y autorización

- **JWT** almacenado en `localStorage` (`tecdiagram_token`).
- Cabecera `Authorization: Bearer` en peticiones a Node (`api.js`).
- Rutas protegidas con `ProtectedRoute` (redirección a `/login`).
- Ruta admin con `AdminRoute` (solo `role === 'superadmin'`).
- UI condicional por rol: profesor / estudiante / invitado.

4.2 Validación de formularios (frontend)

- Campos `required` en login, registro, crear clase, modales.
- Validación de contraseñas coincidentes (registro, recuperar clave).
- Selección obligatoria de rol en registro.
- Mensajes de error vía `AppToast` (no envío silencioso).
- Deshabilitar botón «Publicar en clase» hasta seleccionar un diseño.

4.3 Validación y seguridad en backend (consumida por el frontend)

- Contraseñas hasheadas con **bcrypt** (no se almacenan en texto plano).
- Middleware `authMiddleware` en rutas protegidas de Node.
- Permisos por dueño de clase / miembro / rol en diseños y expulsión de alumnos.
- CORS habilitado en Express para desarrollo con Vite.

4.4 Buenas prácticas aplicadas

- No exponer secretos en el repositorio (`.env` en `.gitignore`).
- Sanitización indirecta: el contenido del diagrama se serializa como JSON (evitar `dangerouslySetInnerHTML` en posts).
- Confirmación antes de eliminar clase, publicación o expulsar estudiante.

5. Repositorio GitHub

5.1 Enlace principal

Repositorio: <https://github.com/MaiteVazquez8/DiagramTec>

5.2 Últimos commits(evidencia de participación)

Sustituir o ampliar con los commits más recientes al momento de la entrega:

Integrante	Enlace a commits	Ejemplos recientes
Santiago Alvarez	Commits	a2115ca Mejoras UI Figma, editor, clases; d6f7524 fix backend/DB

Ramas relevantes frontend: `Frontend` , `main` (tras merges PR #11–#16).

Pull Requests fusionados (referencia): #9, #11, #13, #16 (Frontend), entre otros.

5.3 Evidencia de participación de todos

- Historial de `git shortlog` muestra commits de **cinco integrantes**.
- Uso de ramas `Frontend` , `Backend` , `Diseño` y merges vía Pull Request.
- Commits de merge atribuyen integración colectiva del código.

(Adjuntar captura de GitHub → Insights → Contributors o de la pestaña Commits filtrada por autor.)

6. Organización y documentación del código

6.1 Estructura del repositorio

```
DiagramTec/
├─ frontend/          ← Sector frontend (React + Vite)
│  └─ src/
│  └─ public/
│  └─ FRONTEND.md
│  └─ DOCUMENTACION_EQUIPO_FRONTEND.md (este archivo)
├─ backend/           ← API Node.js
├─ php/               ← Auth PHP
└─ documents/         ← Material de referencia / DB
```

6.2 Convenciones adoptadas

- **Páginas** en `pages/` , una por ruta principal.
- **Componentes** reutilizables en `components/` .
- **Lógica del editor** desacoplada en `design/` (sin dependencia directa de React en modelos).
- **Estilos globales** centralizados en `styles.css` con secciones comentadas (Figma, editor, clases).
- **Cliente HTTP:** `api.js` (Node) y `authApi.js` (PHP) separados.
- Nombres de clases BEM-like: `figma-card` , `class-detail-banner` , `app-toast` .

6.3 Documentación existente

Archivo	Contenido
frontend/FRONTEND.md	Stack, rutas, auth, editor, scripts
frontend/.env.example	Variables de entorno sugeridas
README.md (raíz)	Visión general del proyecto (<i>revisar si está actualizado</i>)

7. Evidencias visuales

Acción del equipo: crear la carpeta `frontend/docs/evidencias/` y subir capturas con nombres descriptivos. Enlazarlas aquí.

7.1 Capturas recomendadas

#	Pantalla	Archivo sugerido
1	Inicio (home)	01-home.png
2	Login / Registro	02-auth.png
3	Mis diseños (grilla Figma)	03-designs.png
4	Editor con diagrama y conexiones	04-editor.png

5	Mis clases (vista profesor)	05-classes-teacher.png
6	Modal crear clase	06-modal-create-class.png
7	Detalle de clase + publicaciones	07-class-detail.png
8	Modal publicar diseño en clase	08-modal-upload-design.png
9	Vista estudiante (unirse a clase)	09-classes-student.png
10	Notificación toast (esquina superior derecha)	10-toast.png
11	Panel superadmin	11-superadmin.png
12	Versión móvil (header + clases)	12-mobile.png

7.2 Funcionalidades destacadas para el informe

- Editor con **zoom**, **pan**, **deshacer/rehacer** y **exportación PDF**.
- **Conexiones** entre formas (una por par de nodos).
- **Clases** con roles diferenciados y código de invitación.
- **Sincronización de sesión** entre pestañas al cerrar sesión.

8. Organización del trabajo

8.1 Tareas (*completar / validar en reunión de cierre*)

Integrante	Tareas realizadas
Santiago Alvarez	Editor (conexiones, touch móvil), detalle de clase, permisos publicar/expulsar, AuthContext, toasts, fixes integración API

8.2 Cronograma

Fase	Estado	Observaciones
Diseño UI (Figma)	Cumplido	Referencia visual de Nayla / equipo
Maquetado pantallas estáticas	Cumplido	Rama Frontend
Editor funcional	Cumplido	Con iteraciones de bugs
Integración API + auth	Cumplido	Node + PHP
Clases y roles	Cumplido	Profesor / estudiante
Pruebas finales y documentación	En curso	Este documento + capturas

(Ajustar fechas según el cronograma académico entregado por la cátedra.)

8.3 Cambios en la organización del grupo

- División por **ramas** (Frontend , Backend , Diseño) para trabajar en paralelo.
- Integración mediante **Pull Requests** hacia main .
- Comunicación para alinear proxy Vite, puertos (4002, 5173) y variables .env .

- Reasignación puntual de fixes de integración (publicar en clase, conexiones) según disponibilidad.

(Documentar aquí cualquier cambio de roles o de integrantes si aplica.)

9. Autoevaluación del proyecto

Respuestas modelo para el informe (el equipo puede editarlas en conjunto).

¿Qué aspectos consideran mejor logrados?

- Interfaz alineada al **diseño Figma** (header, tarjetas, modales, sector clases).
- **Editor de diagramas** funcional con guardado en nube y exportación PDF.
- **Sistema de clases** con roles profesor/estudiante y publicación de diseños.
- Separación clara entre **lógica del editor** (`design/`) y **componentes React**.
- Mejoras de **UX** recientes: toasts unificados, logout multi-pestaña, menús contextuales.

¿Qué mejorarían si tuvieran más tiempo?

- **Tests automatizados** (Vitest / React Testing Library) en modelos del editor y flujos críticos.
- **TypeScript** para tipar formas, conexiones y respuestas de la API.
- Refactor de `styles.css` en módulos CSS o CSS Modules por componente.
- Modo **offline** o borrador local antes de guardar en servidor.
- Accesibilidad (ARIA, navegación por teclado en el editor).
- PWA o empaquetado desktop opcional.

¿Qué conocimientos nuevos adquirieron?

- Desarrollo SPA con **React + Vite** y proxy de desarrollo.
- Integración **dual backend** (Node REST + PHP para auth).
- Manejo de **estado complejo** en canvas (zoom, pan, historial, SVG).
- Autenticación **JWT** y rutas protegidas por rol.
- Trabajo colaborativo con **Git**, ramas y Pull Requests.
- Adaptación de maquetado **Figma** a CSS responsive.
- Depuración en **móvil** (eventos touch, red local).

10. Cómo ejecutar el frontend (anexo)

```
cd DiagramTec/frontend
pnpm install
pnpm run dev
```

- URL local: `http://localhost:5173`
- Requiere backend Node en `4002` y MySQL configurado (ver `backend/.env`).
- Auth PHP: configurar proxy `/php-auth` en `vite.config.js` según Laragon.

Documento generado para entrega académica / informe de equipo. Revisar commits y capturas antes de la fecha límite de entrega.